

Computer Science Engineering BSc (2021) – Final Exam Answers

Question 1

Embedded Systems

Definition

An embedded system is a specialized computer system that combines a processor, memory, and input/output peripheral devices with a dedicated function within a larger mechanical or electronic system. These systems are embedded as part of a complete device and often have real-time computing constraints. Unlike general-purpose computers, embedded systems are task-specific and designed to perform particular operations with high precision and efficiency.

Main Characteristics

Embedded systems range from low complexity with a single microcontroller chip to very high complexity with multiple units, peripherals, and networks. The key characteristics include:

Dedicated Functionality: Embedded systems are designed for specific applications such as automotive control, medical devices, or industrial automation. They execute predefined tasks rather than general computing operations.

Real-Time Operation: Many embedded systems must respond to inputs within strict timing constraints. For example, an airbag deployment system must detect an impact and deploy airbags within milliseconds to be effective.

Resource Constraints: Embedded systems typically operate with limited memory, processing power, and energy resources compared to general-purpose computers. This necessitates efficient software and hardware design.

Reliability and Stability: Since embedded systems often control critical operations, they must operate reliably for extended periods without failure. This is particularly important in safety-critical applications like medical devices and automotive systems.

Interaction with Physical World: Embedded systems interface with the physical environment through sensors and actuators, enabling them to monitor conditions and control mechanical or electrical components.

Control Units of Embedded Systems

Integration Levels The integration level of embedded system control units is described by the complexity of integrated circuits:

Small-Scale Integration (SSI) contains tens of transistors, Medium-Scale Integration (MSI) contains hundreds of transistors, Large-Scale Integration (LSI) contains thousands of transistors, and Very Large-Scale Integration

(VLSI) embeds hundreds of thousands of transistors into a chip. Ultra-Large-Scale Integration (ULSI) places millions of transistors on a single chip.

System on Chip (SoC): A SoC contains a complete system with multiple processors, multipliers, caches, different types of memory, and various peripherals for wired or wireless communication on a single chip. Graphics Processing Units and Digital Signal Processors are often included in SoCs. SoCs can be implemented as Application-Specific Integrated Circuits (ASIC) or using Field-Programmable Gate Arrays (FPGA).

Control Unit Types:

Microcontrollers are the most common control units in embedded systems. They integrate the processor, memory, and I/O interfaces on a single chip, making them cost-effective and power-efficient for dedicated applications.

Microprocessors provide higher computational power and are used in more complex embedded systems. They require external memory and peripheral chips but offer greater flexibility and performance.

Digital Signal Processors (DSPs) are specialized for signal processing tasks, particularly in applications involving audio, video, or sensor data processing. They excel at mathematical operations and real-time data manipulation.

Interfaces and Peripherals Embedded systems communicate with the outside world via peripherals including Serial Communication Interfaces like RS-232, RS-422, RS-485, and Synchronous Serial Interfaces like I2C, SPI, SSC, and ESSI.

Common peripheral interfaces include:

- Universal Asynchronous Receiver-Transmitter (UART) for serial communication
- Inter-Integrated Circuit (I2C) for short-distance communication between chips
- Serial Peripheral Interface (SPI) for high-speed synchronous communication
- Universal Serial Bus (USB) for standardized connectivity
- General Purpose Input/Output (GPIO) pins for flexible interfacing
- Analog-to-Digital Converters (ADC) and Digital-to-Analog Converters (DAC) for signal conversion
- Pulse Width Modulation (PWM) for motor control and analog signal generation

Programming Techniques

Control Loop Programming: Simple control loops call subroutines that manage specific parts of hardware or embedded programming.

Interrupt-Driven Programming: Interrupt controlled systems have a main loop and a secondary loop, where interruptions in the loops trigger tasks. This allows the system to respond immediately to critical events without continuously polling for changes.

Real-Time Operating Systems (RTOS): For complex embedded systems, an RTOS provides task scheduling, inter-task communication, and resource management. This enables efficient multitasking and ensures timing

requirements are met.

Bare-Metal Programming: Some embedded systems run code directly on hardware without an operating system, providing maximum control and minimal overhead.

State Machine Implementation: Many embedded systems are programmed as finite state machines, transitioning between states based on inputs and conditions.

Implementing User Interaction

User interaction in embedded systems can be implemented through various methods depending on the application requirements:

Input Methods:

- Physical buttons and switches for discrete commands
- Touchscreens for intuitive graphical interfaces
- Keypads for numeric or text entry
- Rotary encoders for continuous parameter adjustment
- Voice recognition for hands-free operation
- Gesture recognition using sensors
- Remote controls using infrared or radio frequency communication

Output Methods:

- LED indicators for status display
- LCD or OLED displays for text and graphics
- Seven-segment displays for numeric output
- Audio feedback through buzzers or speakers
- Haptic feedback using vibration motors

Communication Protocols:

- Wireless interfaces (Bluetooth, Wi-Fi, Zigbee) for mobile device connectivity
- Web-based interfaces for remote monitoring and control
- Command-line interfaces for configuration and debugging

The choice of user interface depends on factors such as the application context, user expertise, environmental conditions, and cost constraints. Safety-critical systems often include redundant feedback mechanisms to ensure users are aware of system status.

Program Units

A program unit is a sequence of one or more lines organized as statements, comments, and include directives. Program units form the basic organizational structure of software and can take various forms:

Main Program: The entry point of application execution, containing the primary program logic and coordinating calls to other program units.

Modules: Self-contained units of code that group related functionality, data structures, and procedures. Modules promote code organization and enable separate compilation.

Block Data Program Units: Specialized units for initializing data in common blocks, primarily used in languages like Fortran.

External Function Subprograms: Independent functions that can be called from multiple locations, performing computations and returning values.

External Subroutine Subprograms: Procedures that perform operations but do not return values directly, instead modifying parameters or global state.

Program units enable modular program design, allowing large programs to be divided into manageable components that can be developed, tested, and maintained independently.

Subprograms

A subprogram is a sequence of instructions whose execution is invoked from one or more remote locations in a program, with the expectation that when execution completes, control returns to the instruction after the invocation point. In high-level languages, subprograms are called subroutines, procedures, and functions. In object-oriented languages, they are typically called methods or constructors.

Characteristics of Subprograms:

Modern subprograms can have parameters, allowing data to be passed to them. They can have local variables that exist only during subprogram execution. Functions return values to the calling code, while procedures execute operations without explicit return values.

Benefits:

- **Code Reusability:** Subprograms can be called multiple times from different locations, eliminating code duplication
 - **Modularity:** Breaking code into subprograms improves readability and maintainability
 - **Abstraction:** Subprograms hide implementation details, exposing only necessary interfaces
 - **Testing:** Individual subprograms can be tested independently
-

Parameter Evaluation

Parameter evaluation is the process of mapping formal and actual parameters when a subprogram is called. Formal parameters are defined in the subprogram specification and declared only once. Actual parameters (arguments) are specified in each call to the subprogram. The formal parameter list is determinative for parameter evaluation.

Issues in Parameter Evaluation:

Assigning Actual Parameters to Formal Parameters: Actual parameters are typically assigned to formal parameters based on their relative order in the parameter list. The first argument corresponds to the first formal parameter, the second to the second, and so on. This is called positional parameter passing.

Alternatively, named parameter passing allows specifying the formal parameter name explicitly during the call, making the order irrelevant. For example, in Python:

```
python

def calculate(length, width, height):
    return length * width * height

# Positional
result = calculate(10, 5, 3)

# Named
result = calculate(width=5, height=3, length=10)
```

Number of Parameters: The number of formal parameters is fixed in the subprogram definition. In strict parameter matching, the number of actual parameters must equal the number of formal parameters. However, some languages support default values for parameters. When formal parameters have default values and fewer actual parameters are provided, the missing formal parameters receive their pre-defined default values.

Type Relationships: In strongly-typed languages, the type of each actual parameter must match the type of the corresponding formal parameter exactly. In languages with type coercion, actual parameters may be automatically converted to the type of the formal parameter. Some languages also support polymorphic parameters that can accept multiple types.

Parameter Passing Methods

Parameter passing is the action of transferring actual parameters to formal parameters when a function or subroutine is called. Different methods exist:

Pass by Value: The value of the actual parameter is copied to the formal parameter. The formal and actual parameters occupy different memory locations. Changes to the formal parameter inside the subprogram do not affect the actual parameter. This method is safe because the original data cannot be modified, but it can be inefficient for large data structures.

Example:

```
python

def modify(x):
    x = x + 10
    print(f"Inside function: {x}")

value = 5
modify(value)
print(f"After function: {value}") # Output: 5 (unchanged)
```

Pass by Reference: Both actual and formal parameters refer to the same memory location. The formal parameter is essentially an alias for the actual parameter. Changes made to the formal parameter inside the subprogram directly affect the actual parameter. This is efficient for large data structures but requires caution to avoid unintended modifications.

Example (C++):

```
cpp

void modify(int &x) {
    x = x + 10;
}

int value = 5;
modify(value);
// value is now 15
```

Pass by Value-Result (Copy-In-Copy-Out): The value of the actual parameter is copied to the formal parameter at the call (copy-in). The subprogram works with this copy. Upon return, the value of the formal parameter is copied back to the actual parameter (copy-out). This combines aspects of pass by value and pass by reference.

Pass by Name: The actual parameter is substituted textually for the formal parameter. Evaluation is delayed until the parameter is actually used. This method is rarely used in modern languages due to complexity and potential for confusion.

Block

A block or code block is a structure of source code that is grouped together. Blocks consist of one or more declarations and statements enclosed within delimiters (such as braces `{ }` in C-like languages or keywords like `begin-end` in Pascal).

Purpose of Blocks:

Blocks allow grouping statements to be treated as a single statement. This is essential for control structures like if statements, loops, and functions where multiple statements need to be executed as a unit.

Blocks define scopes for variables, allowing variables with the same name to coexist in different blocks without conflict. Variables declared within a block are typically local to that block.

Example (C):

```
c
{
    int x = 10; // x is local to this block
    {
        int x = 20; // Different x, local to inner block
        printf("%d", x); // Prints 20
    }
    printf("%d", x); // Prints 10
}
```

Scoping and Accessibility

Scope defines where an identifier (variable, constant, function) is recognized and can be used within the program.

Global Scope: A variable defined outside any function has global scope. When the program compiles, global variables are allocated memory at initialization time. Global variables are accessible to all functions in the source code and even in other linked modules. They should be defined at the top of the file before any function definitions.

Example:

```
c

int globalVar = 100; // Global scope

void function1() {
    printf("%d", globalVar); // Accessible here
}

void function2() {
    globalVar = 200; // Can modify here
}
```

Local Scope: Variables defined inside a function or code block have local scope. They are allocated on the stack when the function is called and deallocated when the function returns. Local variables exist only during function execution and are not accessible outside their containing block.

Name Resolution: When a local variable has the same identifier as a global variable, the local variable takes precedence within its scope. The global variable is hidden but not destroyed.

Example:

```
c

int value = 10; // Global

void test() {
    int value = 20; // Local, shadows global
    printf("%d", value); // Prints 20
}
```

Accessibility Levels (in object-oriented languages):

- Public: Accessible from anywhere
 - Private: Accessible only within the defining class
 - Protected: Accessible within the class and its subclasses
 - Package/Internal: Accessible within the same package or module
-

Abstract Data Type

Abstract Data Types (ADT) are objects whose behavior is defined by a set of values and a set of operations. An ADT specifies what operations can be performed but not how these operations are implemented. It does not specify how data is organized in memory or what algorithms are used for implementation.

Characteristics:

- Implementation-independent view of data and operations
- Encapsulation of data and operations
- Information hiding - internal representation is hidden from users
- Interface-based interaction - users interact through defined operations

Common ADTs:

List: An ordered collection of elements with operations like insert, delete, search, and traverse. Lists can be implemented using arrays or linked structures.

Stack: A Last-In-First-Out (LIFO) collection with operations push (add to top), pop (remove from top), and peek (view top). Stacks are used for function call management, expression evaluation, and backtracking algorithms.

Queue: A First-In-First-Out (FIFO) collection with operations enqueue (add to rear), dequeue (remove from front), and peek (view front). Queues are used for scheduling, buffering, and breadth-first search.

Set: An unordered collection of unique elements with operations like add, remove, contains, union, intersection, and difference.

Map/Dictionary: A collection of key-value pairs with operations like put, get, remove, and containsKey.

The same ADT can have multiple implementations. For example, a stack can be implemented using an array or a linked list, but the interface remains the same.

Generic Programming

Generic programming is a style of programming where algorithms and functions are written in terms of placeholder types that are specified later. This allows code to work with all data types rather than just one specific type.

Benefits:

- **Code Reusability:** Generic code can be used with multiple data types without rewriting
- **Type Safety:** Type parameters are checked at compile time
- **Performance:** No runtime overhead compared to type-specific implementations
- **Maintainability:** Single implementation serves multiple types

Implementation:

In C++, templates provide generic programming:

```
cpp

template <typename T>
T maximum(T a, T b) {
    return (a > b) ? a : b;
}

int maxInt = maximum<int>(10, 20);
double maxDouble = maximum<double>(10.5, 20.7);
```

In Java, generics serve the same purpose:

```
java

public <T extends Comparable<T>> T maximum(T a, T b) {
    return (a.compareTo(b) > 0) ? a : b;
}
```

Generic collections like `ArrayList<T>`, `HashMap<K,V>`, and `Stack<T>` allow type-safe data structures that work with any object type.

I/O Tools and File Handling

Computers store data in files including plain text, images, and other formats. Programming languages provide tools for file access, creation, and modification.

File Operations:

Input (Reading): Retrieving data from a file **Output (Writing):** Storing data to a file

File Access Modes:

Read-Only Mode: Allows only retrieving data from the file. The file must exist, and cannot be modified.

Write-Only Mode: Allows writing data to the file. Creates a new file or overwrites existing content.

Read-Write Mode: Permits both reading and writing operations.

Append Mode: Allows writing to the end of the file without overwriting existing content.

File Handling Process:

1. Open the file using appropriate mode and obtain a file object/handle
2. Perform read or write operations using file methods
3. Close the file to free allocated resources

Python example:

```
python

# Writing to a file
file = open("data.txt", "w")
file.write("Hello, World!")
file.close()

# Reading from a file
file = open("data.txt", "r")
content = file.read()
file.close()

# Using context manager (recommended)
with open("data.txt", "r") as file:
    content = file.read()

# File automatically closed after block
```

Binary file operations handle non-text data like images, videos, and serialized objects. Text file operations process human-readable text with character encoding considerations.

Exception Handling

Exception handling is the process of responding to exceptions or errors that occur during program execution. An exception disrupts normal program flow and, without handling, would cause the program to crash.

Purpose:

- Gracefully handle runtime errors
- Maintain program stability
- Provide meaningful error messages
- Separate error-handling code from normal logic
- Enable recovery from errors

Implementation:

Most languages use try-catch (or try-except) blocks:

```
python

try:
    # Code that might raise an exception
    value = int(input("Enter a number: "))
    result = 100 / value
    print(f"Result: {result}")
except ValueError:
    print("Invalid input! Please enter a number.")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:
    print(f"An error occurred: {e}")
finally:
    print("Execution completed.") # Always executes
```

Common Exception Types:

- ArithmeticError: Division by zero, overflow
- IndexError: Array/list index out of bounds
- NullPointerException: Accessing null reference
- FileNotFoundError: Attempting to open non-existent file

- IOError: Input/output operation failure

Custom Exceptions: Programs can define custom exception types for application-specific errors.

Parallel Programming

Parallel programming uses multiple processors or threads to execute parts of a program simultaneously. The program is divided into smaller tasks that execute concurrently on multiple threads or cores.

Key Concepts:

Thread: A lightweight unit of execution within a process. Multiple threads share the same memory space but execute independently.

Process: An independent program instance with its own memory space. Processes provide stronger isolation than threads.

Concurrency: Multiple tasks making progress during overlapping time periods (not necessarily simultaneously).

Parallelism: Multiple tasks executing simultaneously on multiple processors/cores.

Advantages:

- Improved performance through utilizing multiple cores
- Better resource utilization
- Responsive user interfaces (background processing)
- Handling multiple simultaneous operations

Challenges:

- Race conditions: Multiple threads accessing shared data
- Deadlocks: Circular dependencies preventing progress
- Synchronization overhead
- Debugging complexity

Synchronization Mechanisms:

- Mutex (Mutual Exclusion): Ensures only one thread accesses critical section
- Semaphore: Controls access to resource pool
- Lock: Protects shared data
- Atomic operations: Indivisible operations on shared variables

Example (Python):

```
python

import threading

def task(name):
    for i in range(5):
        print(f'{name}: {i}')

thread1 = threading.Thread(target=task, args=("Thread-1",))
thread2 = threading.Thread(target=task, args=("Thread-2",))

thread1.start()
thread2.start()

thread1.join()
thread2.join()
```

Modern processors feature multiple cores with Simultaneous Multithreading (SMT), allowing each core to execute multiple threads concurrently. Effective parallel programming requires careful design to ensure tasks are independent and synchronization is properly managed.

Question 2

Synthesis of Continuous Time Control Systems

Control system synthesis is the process of designing a controller that achieves desired system behavior. For continuous time systems, this involves determining controller parameters and structures to meet performance specifications.

Design Steps:

1. **System Modeling:** Develop mathematical model of the plant (system to be controlled) using differential equations or transfer functions
2. **Specification:** Define desired performance criteria (settling time, overshoot, steady-state error)
3. **Controller Design:** Choose controller type (P, PI, PID, lead-lag) and determine parameters
4. **Analysis:** Verify system meets specifications using frequency or time domain analysis
5. **Implementation:** Realize controller in hardware or software

Common Controller Types:

Proportional (P) Controller: Output proportional to error

- Transfer function: K_p
- Fast response but steady-state error

Proportional-Integral (PI) Controller: Eliminates steady-state error

- Transfer function: $K_p + K_i/s$
- Zero steady-state error but slower response

Proportional-Integral-Derivative (PID) Controller: Combines all three actions

- Transfer function: $K_p + K_i/s + K_d \cdot s$
- Fast response, zero steady-state error, improved stability

Design Methods:

- Root locus: Graphical method showing closed-loop pole locations as gain varies
 - Frequency response: Using Bode plots and Nyquist diagrams
 - State-space: Modern control using state feedback and observers
-

Gain and Phase Margin

Gain margin and phase margin are frequency domain measures of system stability. They indicate how close the system is to instability and provide robustness metrics.

Gain Margin (GM): The gain margin is the amount by which the system gain can be increased before the system becomes unstable. It is measured at the phase crossover frequency (where phase is -180°).

$GM = 1 / |G(j\omega)|$ at phase crossover frequency (in linear terms)

$GM \text{ (dB)} = -20 \log |G(j\omega)|$

A gain margin greater than 0 dB indicates stability. Typical design targets are $GM > 6 \text{ dB}$ for adequate robustness.

Phase Margin (PM): The phase margin is the amount of additional phase lag at the gain crossover frequency (where $|G(j\omega)| = 1$ or 0 dB) that would make the system unstable.

$PM = 180^\circ + \angle G(j\omega)$ at gain crossover frequency

A positive phase margin indicates stability. Typical design targets are $PM > 30^\circ$ for adequate damping, with $PM \approx 60^\circ$ providing good transient response.

Interpretation:

- Larger margins indicate more robust, stable systems

- Small margins suggest sensitivity to parameter variations
- Both margins should be positive for stable systems
- Negative margins indicate unstable systems

Reading from Bode Plots:

1. Locate gain crossover frequency (magnitude = 0 dB)
 2. Read phase at this frequency and calculate PM = phase + 180°
 3. Locate phase crossover frequency (phase = -180°)
 4. Read magnitude at this frequency and calculate GM = -magnitude (dB)
-

Linear Systems and Their Description

Linear systems obey the principles of superposition and homogeneity. If $y_1(t)$ is the response to input $x_1(t)$ and $y_2(t)$ is the response to $x_2(t)$, then the response to $a \cdot x_1(t) + b \cdot x_2(t)$ is $a \cdot y_1(t) + b \cdot y_2(t)$.

Time-Domain Description:

Linear time-invariant (LTI) systems in time domain are described by differential equations:

For a second-order system:

$$a_2(d^2y/dt^2) + a_1(dy/dt) + a_0y = b_1(dx/dt) + b_0x$$

The solution consists of:

- Natural response (homogeneous solution): System's inherent behavior
- Forced response (particular solution): Response to input

State-Space Representation:

$$\begin{aligned} dx/dt &= Ax + Bu \quad (\text{state equation}) \\ y &= Cx + Du \quad (\text{output equation}) \end{aligned}$$

Where x is state vector, u is input, y is output, and A, B, C, D are system matrices.

Frequency-Domain Description:

Taking the Laplace transform converts differential equations to algebraic equations.

Transfer Function: Ratio of output to input in Laplace domain

$$G(s) = Y(s)/X(s) = (b_n s^n + \dots + b_1 s + b_0)/(a_m s^m + \dots + a_1 s + a_0)$$

Frequency Response: System response to sinusoidal inputs

- Obtained by substituting $s = j\omega$ in transfer function
- $G(j\omega)$ is a complex number with magnitude and phase
- Magnitude $|G(j\omega)|$ represents gain at frequency ω
- Phase $\angle G(j\omega)$ represents phase shift at frequency ω

Bode Plots: Graphical representation of frequency response

- Magnitude plot: $20\log|G(j\omega)|$ vs $\log(\omega)$ in dB
- Phase plot: $\angle G(j\omega)$ vs $\log(\omega)$ in degrees

Nyquist Plot: Polar plot of $G(j\omega)$ as ω varies from 0 to ∞

The frequency domain is particularly useful for:

- Stability analysis
- Controller design
- Understanding system behavior across frequency ranges

Signal Transfer in Control Systems

Signal transfer describes how signals propagate through a control system from input to output.

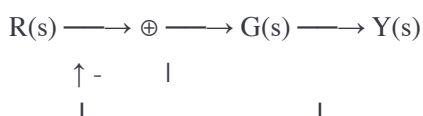
Block Diagram Representation: Control systems are represented using blocks (transfer functions), summing junctions, and signal paths.

Basic Elements:

Forward Path: Signal path from input through controller and plant to output **Feedback Path:** Signal path from output back to input for comparison

Transfer Functions:

For a unity feedback system:



Closed-loop transfer function:

$$Y(s)/R(s) = G(s)/(1 + G(s))$$

Signal Flow Graphs: Alternative representation using nodes (signals) and branches (gains)

Mason's Gain Formula: Systematic method for finding overall transfer function:

$$G(s) = (\sum P_k \Delta_k) / \Delta$$

Where P_k are forward path gains, Δ is graph determinant, Δ_k is cofactor of path k .

Cascade Connection: Output of one block is input to next

- Overall transfer function: $G(s) = G_1(s) \cdot G_2(s)$

Parallel Connection: Same input to multiple blocks, outputs summed

- Overall transfer function: $G(s) = G_1(s) + G_2(s)$

Feedback Connection: Output fed back to input

- Closed-loop TF: $G(s) = G_1(s) / (1 \pm G_1(s) \cdot G_2(s))$

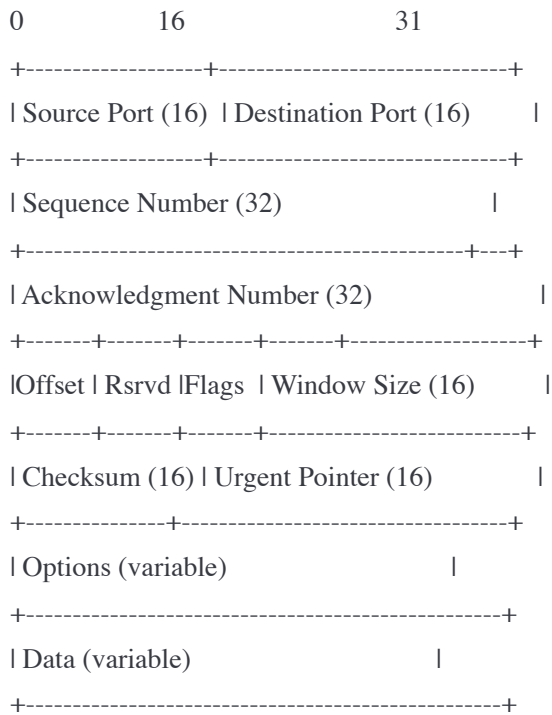
Understanding signal transfer is essential for analyzing system behavior and designing controllers.

TCP and UDP Protocol Data Units

TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are transport layer protocols in the Internet Protocol Suite.

TCP Protocol Data Unit (Segment):

TCP segments consist of a header and data. The TCP header contains:

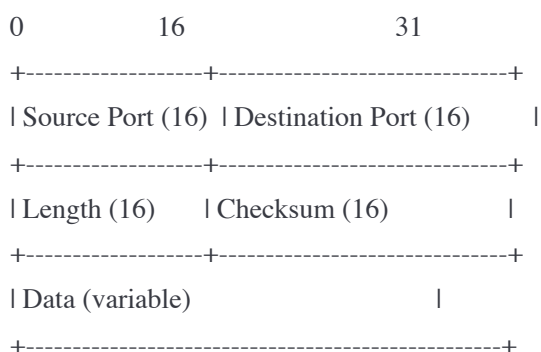


Key TCP Header Fields:

- **Source/Destination Port** (16 bits each): Identify sending and receiving applications
- **Sequence Number** (32 bits): Byte number of first data byte in this segment
- **Acknowledgment Number** (32 bits): Next sequence number expected by receiver
- **Data Offset** (4 bits): Header length in 32-bit words
- **Flags** (9 bits): Control flags including SYN, ACK, FIN, RST, PSH, URG
- **Window Size** (16 bits): Flow control, indicates receiver buffer space
- **Checksum** (16 bits): Error detection for header and data
- **Urgent Pointer** (16 bits): Offset of urgent data when URG flag set
- **Options**: Variable length, includes Maximum Segment Size, Window Scaling, Timestamps

UDP Protocol Data Unit (Datagram):

UDP datagrams have a simpler structure:



UDP Header Fields:

- **Source Port** (16 bits): Optional, identifies sending application
- **Destination Port** (16 bits): Identifies receiving application
- **Length** (16 bits): Total length of UDP datagram (header + data)
- **Checksum** (16 bits): Optional error detection (mandatory in IPv6)

TCP Characteristics:

- **Connection-Oriented:** Three-way handshake (SYN, SYN-ACK, ACK) establishes connection before data transfer
- **Reliable:** Guarantees data delivery through acknowledgments and retransmissions
- **Ordered:** Sequence numbers ensure data arrives in order
- **Flow Control:** Window mechanism prevents receiver overflow
- **Congestion Control:** Adjusts transmission rate based on network conditions
- **Error Detection:** Checksum detects corrupted segments
- **Full-Duplex:** Simultaneous bidirectional communication

UDP Characteristics:

- **Connectionless:** No connection establishment, sends data immediately
- **Unreliable:** No delivery guarantees, packets may be lost
- **Unordered:** No sequence numbers, packets may arrive out of order
- **No Flow Control:** No mechanism to prevent receiver overflow
- **No Congestion Control:** Sends at application-specified rate
- **Minimal Error Detection:** Optional checksum
- **Low Overhead:** Small header (8 bytes vs 20+ bytes for TCP)

Differences Summary:

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable	Unreliable
Ordering	Ordered delivery	No ordering
Speed	Slower (overhead)	Faster (minimal overhead)
Header Size	20-60 bytes	8 bytes
Use Cases	Web, email, file transfer	Streaming, gaming, DNS
Flow Control	Yes	No
Error Recovery	Yes (retransmission)	No

Use Cases:

TCP is used for applications requiring reliable data transfer: HTTP/HTTPS (web), SMTP/POP3/IMAP (email), FTP (file transfer), SSH (remote access).

UDP is used for applications prioritizing speed over reliability: DNS (name resolution), DHCP (IP configuration), streaming media, online gaming, VoIP, SNMP (network management).

Question 3

Combinational Logic Design

Combinational logic circuits are digital circuits where outputs depend only on current inputs, with no memory of past states. The output is a pure function of present inputs.

Characteristics:

- No feedback paths
- No storage elements (flip-flops, latches)
- Output determined solely by current input combination
- Described by Boolean functions

Design Process:

1. Define problem specifications
2. Determine required number of inputs and outputs

3. Derive truth table
 4. Obtain simplified Boolean functions (K-maps, algebraic methods)
 5. Draw logic diagram
 6. Verify design
-

Multiplexers and Demultiplex